

CODE	OUTPUT
<pre> #include <stdio.h> #define SIZE 8 #define EMPTY -1 // Primary hash function int hash1(int key) { return key % SIZE; } // Second hash function for double hashing int hash2(int key) { return 1 + 2 * (key % 4); } // Initialize hash table void initialize(int table[]) { for (int i = 0; i < SIZE; i++) table[i] = EMPTY; } // Display hash table void displayTable(int table[]) { printf("\nIndex\tValue\n"); printf("-----\n"); for (int i = 0; i < SIZE; i++) { if (table[i] == EMPTY) printf("%d\tEMPTY\n", i); else printf("%d\t%d\n", i, table[i]); } } // Linear Probing void linearProbing(int keys[], int n, int table[]) { initialize(table); printf("\n===== LINEAR PROBING\n"); for (int i = 0; i < n; i++) { int key = keys[i]; int index = hash1(key); int probes = 1; printf("\nKey %d: %d", key, index); while (table[index] != EMPTY) { index = (index + 1) % SIZE; probes++; printf(" -> %d", index); } table[index] = key; printf("\n Inserted at %d", index); printf("\n Probes = %d\n", probes); } printf("\nFinal Hash Table:"); </pre>	<pre> ===== ===== RAILWAY BOOKING HASHING SYSTEM ===== ===== Input Booking IDs: 23 43 13 33 53 63 73 Hash Table Size = 8 Primary Hash Function: h(k) = k % 8 Load Factor = 7/8 = 0.88 ===== LINEAR PROBING ===== Key 23: 7 Inserted at 7 Probes = 1 Key 43: 3 Inserted at 3 Probes = 1 Key 13: 5 Inserted at 5 Probes = 1 Key 33: 1 Inserted at 1 Probes = 1 Key 53: 5 -> 6 Inserted at 6 Probes = 2 Key 63: 7 -> 0 Inserted at 0 Probes = 2 Key 73: 1 -> 2 Inserted at 2 Probes = 2 Final Hash Table: Index Value ----- 0 63 1 33 2 73 3 43 4 EMPTY 5 13 6 53 7 23 ===== QUADRATIC PROBING ===== Key 23: 7 Inserted at 7 Probes = 1 Key 43: 3 Inserted at 3 Probes = 1 Key 13: 5 Inserted at 5 Probes = 1 Key 33: 1 Inserted at 1 Probes = 1 Key 53: 5 -> 6 Inserted at 6 Probes = 2 Key 63: 7 -> 0 Inserted at 0 Probes = 2 </pre>

```

displayTable(table);
}
// Quadratic Probing
void quadraticProbing(int keys[], int n, int table[]) {
    initialize(table);
    printf("\n===== QUADRATIC PROBING\n");
    for (int i = 0; i < n; i++) {
        int key = keys[i];
        int h = hash1(key);
        int index = h;
        int j = 0;
        int probes = 1;
        printf("\nKey %d: %d", key, index);
        while (table[index] != EMPTY) {
            j++;
            index = (h + j * j) % SIZE;
            probes++;
            printf(" -> %d", index);
        }
        table[index] = key;
        printf("\nInserted at %d", index);
        printf("\nProbes = %d\n", probes);
    }
    printf("\nFinal Hash Table:");
    displayTable(table);
}
// Double Hashing
void doubleHashing(int keys[], int n, int table[]) {
    initialize(table);
    printf("\n===== DOUBLE HASHING\n");
    for (int i = 0; i < n; i++) {
        int key = keys[i];
        int h1 = hash1(key);
        int h2 = hash2(key);
        int index = h1;
        int j = 0;
        int probes = 1;
        printf("\nKey %d: h1=%d, h2=%d", key, h1, h2);
        printf("\nProbe sequence: %d", index);
        while (table[index] != EMPTY) {
            j++;
            index = (h1 + j * h2) % SIZE;
            probes++;
            printf(" -> %d", index);
        }
        table[index] = key;
        printf("\nInserted at %d", index);
        printf("\nProbes = %d\n", probes);
    }
    printf("\nFinal Hash Table:");
    displayTable(table);
}
// Search using Linear Probing
int searchLinear(int table[], int key) {
    int index = hash1(key);

```

Key 73: 1 -> 2 Inserted at 2 Probes = 2

Final Hash Table:

Index	Value
0	63
1	33
2	73
3	43
4	EMPTY
5	13
6	53
7	23

0 63

1 33

2 73

3 43

4 EMPTY

5 13

6 53

7 23

===== DOUBLE HASHING

=====

Key 23: h1=7, h2=7

Probe sequence: 7 Inserted at 7 Probes = 1

Key 43: h1=3, h2=7

Probe sequence: 3 Inserted at 3 Probes = 1

Key 13: h1=5, h2=3

Probe sequence: 5 Inserted at 5 Probes = 1

Key 33: h1=1, h2=3

Probe sequence: 1 Inserted at 1 Probes = 1

Key 53: h1=5, h2=3

Probe sequence: 5 -> 0 Inserted at 0
Probes = 2

Key 63: h1=7, h2=7

Probe sequence: 7 -> 6 Inserted at 6
Probes = 2

Key 73: h1=1, h2=3

Probe sequence: 1 -> 4 Inserted at 4
Probes = 2

Final Hash Table:

Index	Value
0	53
1	33
2	EMPTY
3	43
4	73
5	13
6	63
7	23

0 53

1 33

2 EMPTY

3 43

4 73

5 13

6 63

7 23

=====

<pre> int probes = 1; while (table[index] != EMPTY && probes <= SIZE) { if (table[index] == key) return probes; index = (index + 1) % SIZE; probes++; } return -probes; } // Search using Quadratic Probing int searchQuadratic(int table[], int key) { int h = hash1(key); int index = h; int j = 0; int probes = 1; while (table[index] != EMPTY && probes <= SIZE) { if (table[index] == key) return probes; j++; index = (h + j * j) % SIZE; probes++; } return -probes; } // Search using Double Hashing int searchDouble(int table[], int key) { int h1 = hash1(key); int h2 = hash2(key); int index = h1; int j = 0; int probes = 1; while (table[index] != EMPTY && probes <= SIZE) { if (table[index] == key) return probes; j++; index = (h1 + j * h2) % SIZE; probes++; } return -probes; } // Display search result void displaySearchResult(int key, int probes) { if (probes > 0) printf("Key %d: FOUND, Probes = %d\n", key, probes); else printf("Key %d: NOT FOUND, Probes = %d\n", key, - probes); } int main() { int keys[] = {23, 43, 13, 33, 53, 63, 73}; int n = sizeof(keys) / sizeof(keys[0]); int linearTable[SIZE]; int quadraticTable[SIZE]; int doubleTable[SIZE]; </pre>	<pre> SEARCH RESULTS ===== ===== --- Linear Probing --- Key 53: FOUND, Probes = 2 Key 63: FOUND, Probes = 2 Key 73: FOUND, Probes = 2 Key 80: NOT FOUND, Probes = 5 --- Quadratic Probing --- Key 53: FOUND, Probes = 2 Key 63: FOUND, Probes = 2 Key 73: FOUND, Probes = 2 Key 80: NOT FOUND, Probes = 3 --- Double Hashing --- Key 53: FOUND, Probes = 2 Key 63: FOUND, Probes = 2 Key 73: FOUND, Probes = 2 Key 80: NOT FOUND, Probes = 3 ===== ===== END ===== ===== </pre>
--	---

```

printf("=====\n");
printf("    RAILWAY BOOKING HASHING SYSTEM\n");
printf("=====\n");
printf("\nInput Booking IDs:\n");
for (int i = 0; i < n; i++)
    printf("%d ", keys[i]);
printf("\n\nHash Table Size = %d", SIZE);
printf("\nPrimary Hash Function: h(k) = k %%% %d",
SIZE);
printf("\nLoad Factor = %d/%d = %.2f\n", n, SIZE,
(float)n / SIZE);
// Perform insertions
linearProbing(keys, n, linearTable);
quadraticProbing(keys, n, quadraticTable);
doubleHashing(keys, n, doubleTable);
// Search operations
int searchKeys[] = {53, 63, 73, 80};
int searchCount = 4;
printf("\n\n=====\n");
printf("    SEARCH RESULTS\n");
printf("=====\n");

printf("\n--- Linear Probing ---\n");

for (int i = 0; i < searchCount; i++)
    displaySearchResult(searchKeys[i],
        searchLinear(linearTable, searchKeys[i]));

printf("\n--- Quadratic Probing ---\n");

for (int i = 0; i < searchCount; i++)
    displaySearchResult(searchKeys[i],
        searchQuadratic(quadraticTable,
searchKeys[i]));

printf("\n--- Double Hashing ---\n");

for (int i = 0; i < searchCount; i++)
    displaySearchResult(searchKeys[i],
        searchDouble(doubleTable, searchKeys[i]));

printf("\n\n=====\n");
printf("    END\n");
printf("=====\n");
return 0;
}

```